Ingeniería de Software I

Ciclos de Vida

Mgtr. Ing. Federico Botti

Agenda - Ciclos de Vida

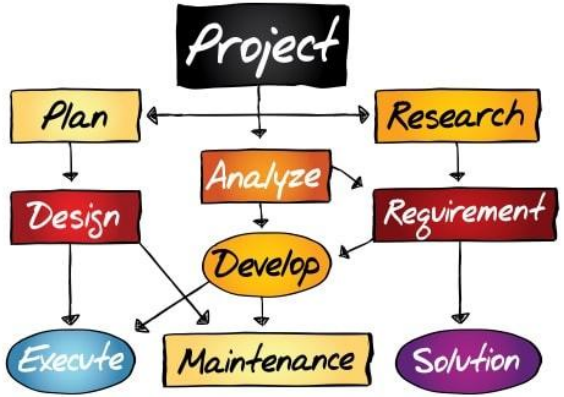
ITBA
Instituto Tecnológico de Buenos Aires

1. Introducción
    a. Qué son?
    b. Por qué?
2. Modelos de Ciclo de Vida Prescriptivos
    a. Cascada
    b. Prototipado
    c. Evolutivo
    d. Incremental
    e. Espiral
3. Proceso Unificado (RUP)
4. Links de interés

Ciclos de Vida - ¿Qué es?

*Proceso **sistemático y estandarizado** por el cual un producto de software es desarrollado y mantenido a lo largo del tiempo.*

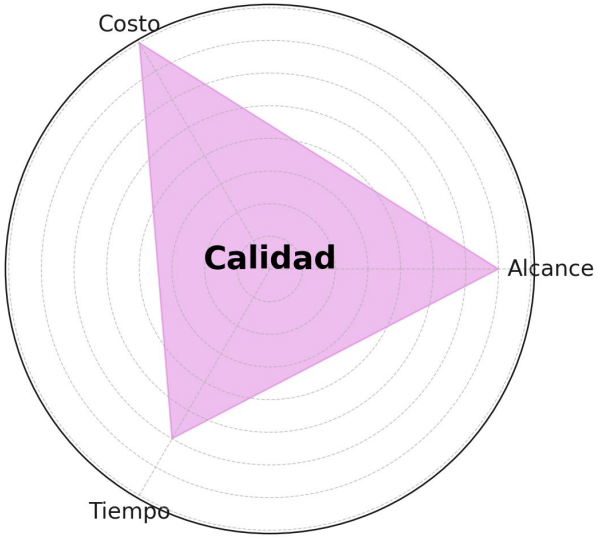
***Define un marco común** para los procesos del ciclo de vida de los programas informáticos, con una terminología bien definida, a la que pueda remitirse la industria del software. Contiene **procesos, actividades y tareas aplicables** durante **la adquisición, el suministro, el desarrollo, el funcionamiento, el mantenimiento o la eliminación de sistemas, productos y servicios informáticos**. Estos procesos del ciclo de vida se llevan a cabo mediante la participación de los interesados, con el objetivo final de lograr **la satisfacción del cliente**".*



Fuente: ISO/IEC/IEEE 12207:2017

Ciclos de Vida - Por qué?

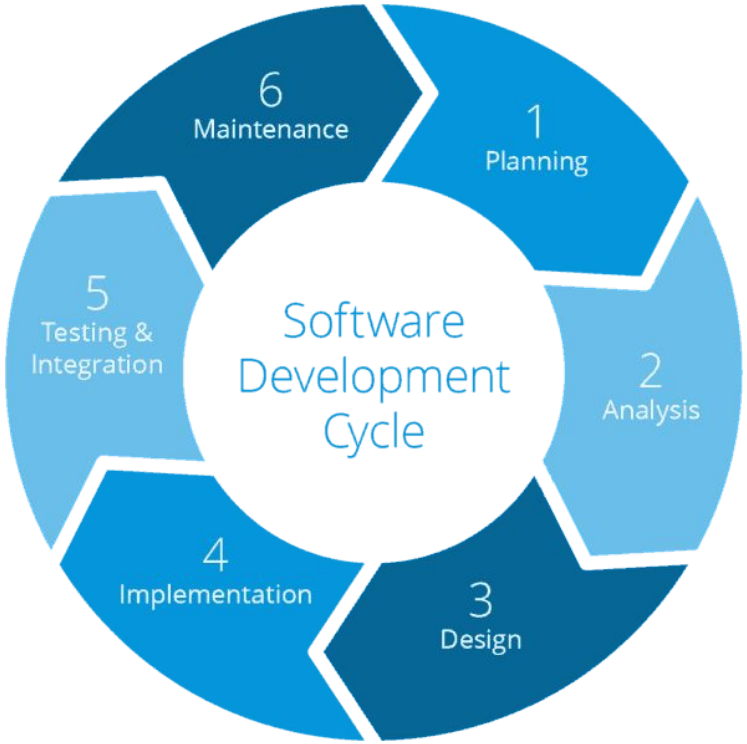
- Construir Software de Alta Calidad
- Predecir la etapa del ciclo de desarrollo
  - RRHH
  - Hardware y Arquitectura
- Minimizar errores
- Minimizar Costos
- Reutilizable y fácilmente mantenible
- Cumplir con los requisitos del cliente
- Escalabilidad



ITBA
Instituto Tecnológico de Buenos Aires

Modelos de Ciclos de Vida

Modelos de Ciclos de Vida



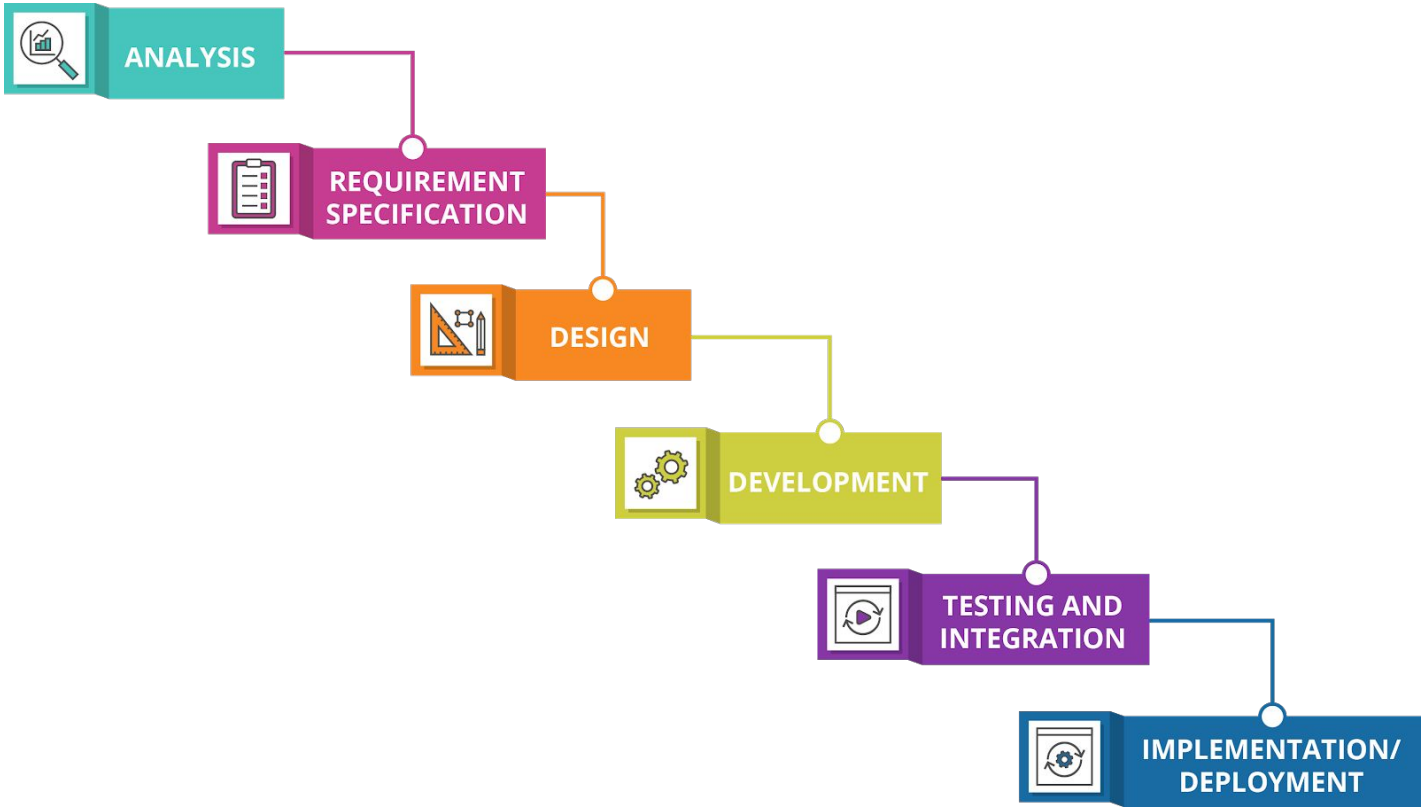
Prescriptivos

- Cascada
- Prototipado
- Evolutivo
- Incremental
- Espiral
- Proceso Unificado

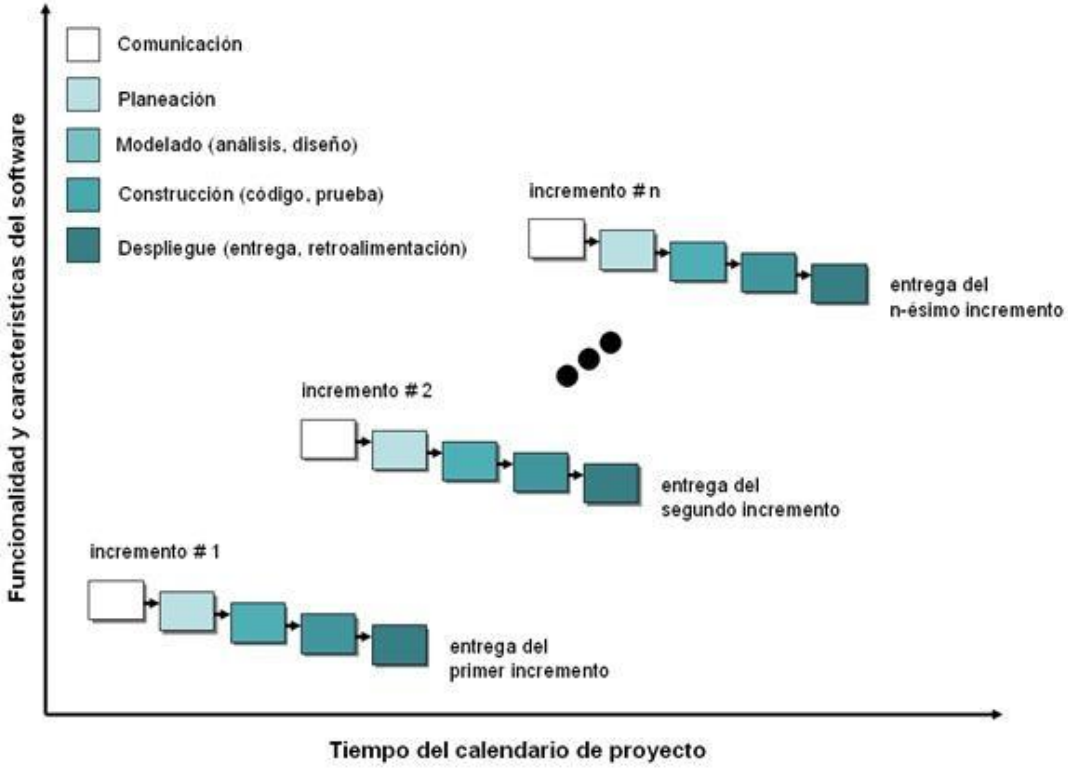
Adaptativos - Ágiles

- Extreme Programming
- Scrum
- Kanban
- Lean Startup

Modelo Cascada



Modelo Incremental

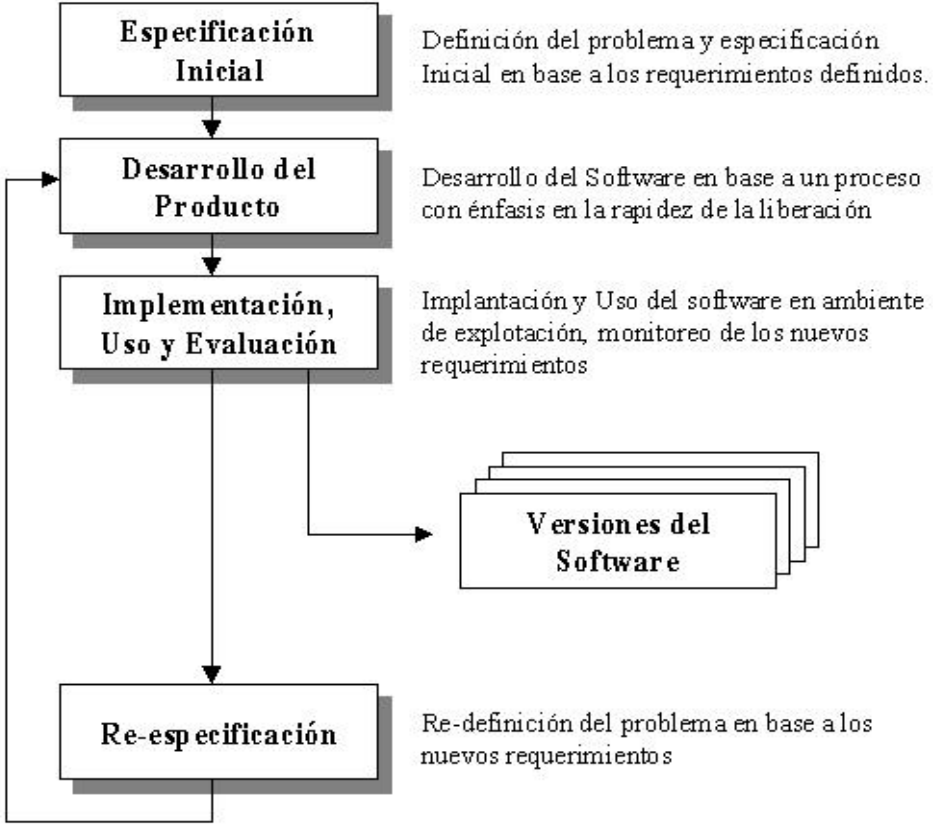


Cómo funciona:

- Se define el **sistema completo** y se lo divide en **funcionalidades**.
- Cada incremento sigue **todas las fases del ciclo de vida** (análisis → diseño → codificación → pruebas → entrega).
- Se entrega al usuario **funcionalidad completa por partes.**
-

Ej: Sobre Gmail: Google Meet, Google Classroom, Google Jamboard.

Modelo Evolutivo



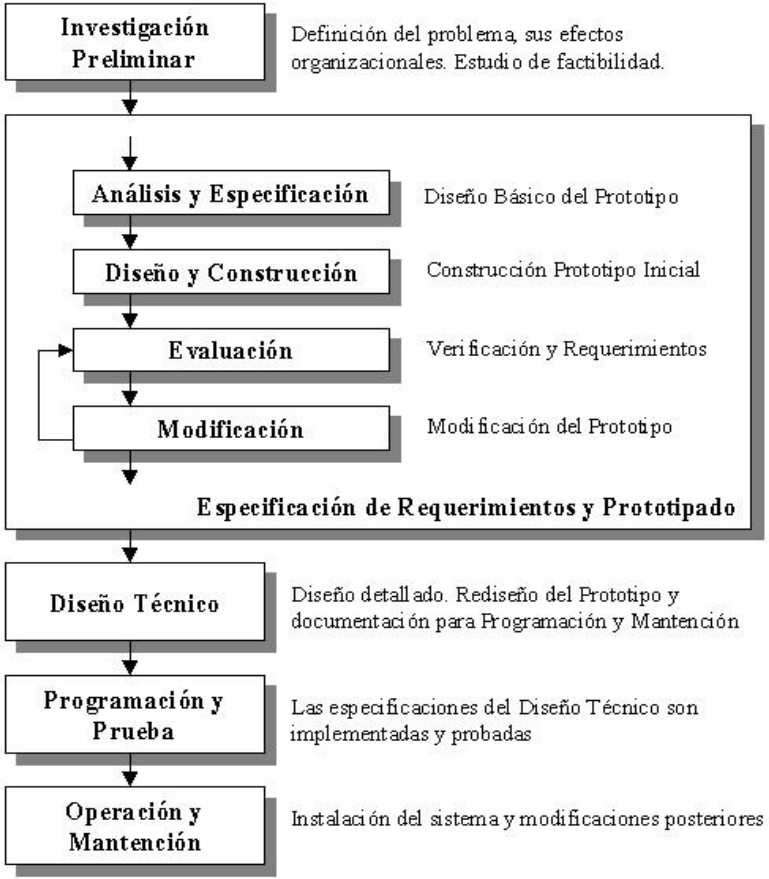
Cómo funciona:

- Se define un **conjunto mínimo de requerimientos**.
- Se desarrolla una **primera versión o prototipo**.
- Se entrega al usuario y se **recoge feedback**.
- Se **planifican mejoras** y se desarrolla una nueva versión.
- El ciclo se **repite** hasta alcanzar el producto final.

Modelo de desarrollo que , a diferencia del de Prototipos, busca reemplazar el viejo sistema con uno nuevo que tendría la propiedad de satisfacer los nuevos requerimientos lo más rápido posible. El desarrollo evolutivo asume que los requerimientos están sujetos a cambios continuos y que la estrategia para enfrentar aquello pasa por un reflejo, también continuo de aquellos cambios.

Ej: Windows, Office, Ubuntu.

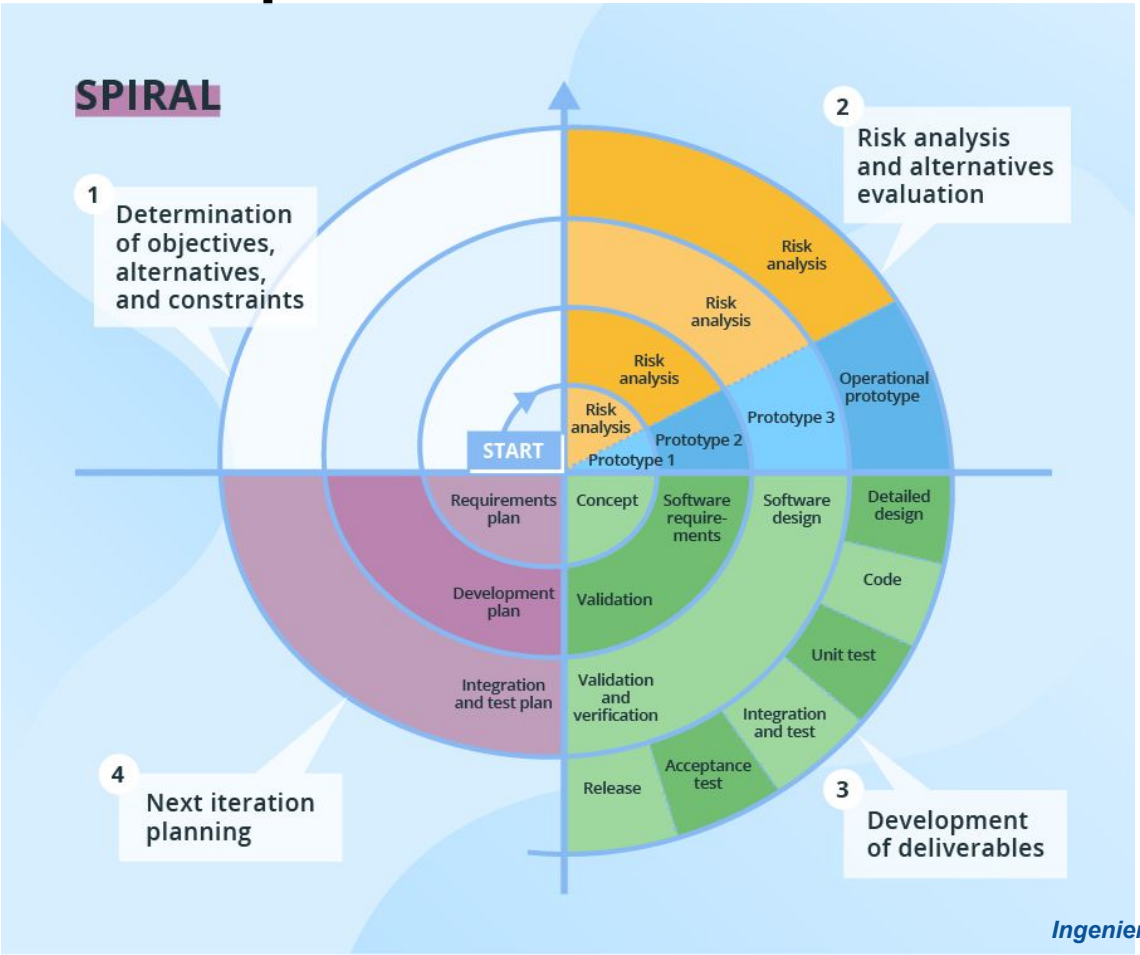
Modelo Prototipado



Modelo que pone énfasis en la etapa de Especificación de Requerimientos a través de la construcción de prototipos que aproximan al usuario a la idea final del sistema, con objeto de poder clarificar los requerimientos.

Ej: Nueva interfaz de Gmail / Facebook / Etc.

Modelo Espiral



- *Prototipado*
- *Evolutivo*
- *Iterativo*
- *Incremental*

Proceso Unificado

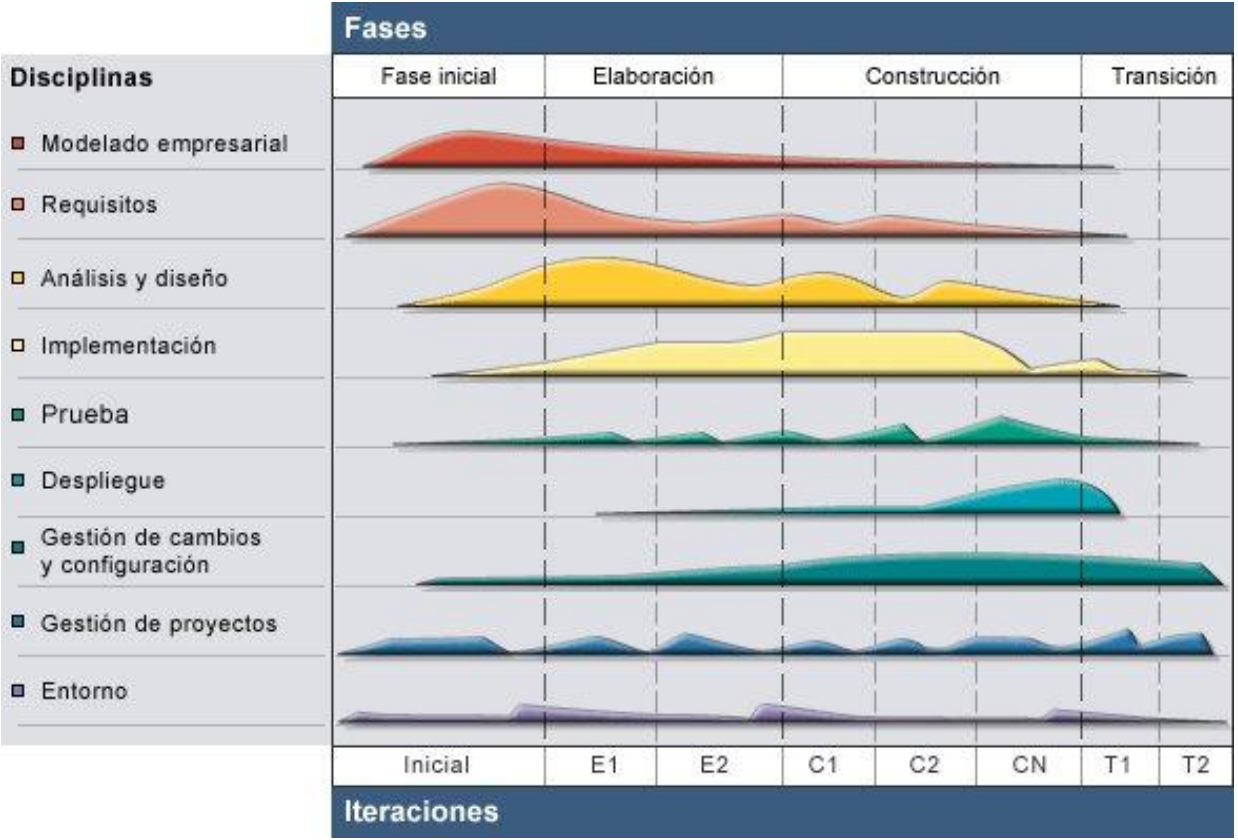
Modelo Proceso Unificado

- UP es un framework de proceso detallado
- UP define aproximadamente unos 50 opcionales artefactos (non-software work products), tales como
  - Visión, Lista de Riesgos, etc
  - En RUP están organizados dentro de las **disciplinas**, y los encontramos como **templates**
  - La regla a seguir con respecto a la creación de estos documentos es “menos es mejor”
  - Esta única adaptación es llamada “**Development Case**” del proyecto

RUP®
Rational Unified Process®

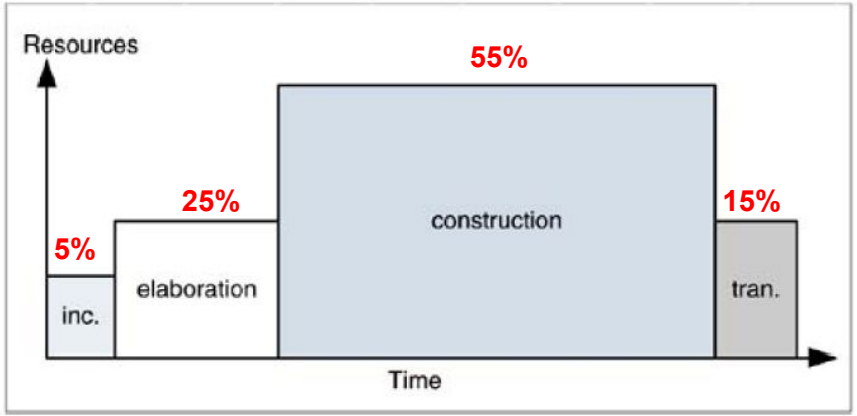
Referencia: RUP

Modelo Proceso Unificado



Referencia: RUP Artifacts

Modelo Proceso Unificado - Esfuerzo



- Los recursos y el staff son variables
  - Elaboration:Equipos pequeños, cohesivos, co-localizados, que forman el corazón del desarrollo
  - Construction:Equipos grandes, y más desarrollo en paralelo

Referencia: RUP Artifacts

Modelo Proceso Unificado - Mejores Prácticas

- **Iteraciones TimeBoxed**
  - Recomendadas entre 2 y 6 semanas
- **Programando los elementos de alto riesgo, y alto valor**
  - Incentivar a la reutilización de componentes
- **Verificación de calidad continuada**
  - Uso de técnicas de integración y testeo continuo, como XP
- **Modelado visual**
  - Esquemas en una pizarra durante 1 hora
  - Esquemas basados en UML
- **Administración de requerimientos**
  - Encontrar (req. Funcionales a través de workshops)
  - Organizar (según prioridades, atributos, riesgos, …)
  - y seguir los requerimientos (según su estado – asignado, en curso, ….)
- **Administración del cambio**
  - A través de la administración de la config. y del control de versión
  - El uso de un protocolo de solicitud de cambios
  - Releases al final de cada iteración
-

Referencia: UML, Herramienta para UML

Modelo Proceso Unificado - Errores comunes

- **Iteraciones**
  - demasiado largas
  - Cada iteración No finaliza con un release en producción
- **El objetivo de la fase elaboración**
  - Crear un prototipo desechable
- **Planeamiento**
  - Predictivo No Adaptativo
  - Diferir el testeo mayor hasta cerca de finalizar el proyecto
- **Documentación**
  - Hacer cantidades de diagramas de UML
  - Realizar un **"Development Case**" demasiado complejo, con muchos work products.
  - El Software Architecture Document (**SAD**) finalizado antes de terminar la elaboración

Links de interés

Ciclos de vida - Links de interés

- RUP
- RUP Artifacts
- UML
- Herramienta para UML